COMOONLY · BEGINNER'S GUIDE

How to read your own code

A map of the folders, the one pattern everything follows, and a walk through a real click from button to database. No experience needed.

1. The big picture: three parts

Your app is three separate pieces that talk to each other. Almost every question ("where is this?") starts with: **which of the three is it in?**

The website

What people see and click: screens, buttons, colours. Runs *inside the visitor's browser*.

Folder: `src/`

⇄

The server

The brain: checks who you are, what you're allowed to do, and does the work. Runs *on your Mac*.

Folder: `server/`

⇄

The database

The memory: every account, group, message and event, in one file.

File: `server/data/comoonly.db`

The website **never** touches the database directly. It always asks the server, through addresses that start with `/api/` (for example `/api/groups`). The server decides whether to say yes.

An everyday comparison: a restaurant. The website is the dining room and the menu. The server is the kitchen: a *receptionist* takes the order (routes), a *bouncer* checks it makes sense (schema), and the *chef* cooks it (service). The database is the pantry. Guests never walk into the pantry.

The languages you'll see

File ending	What it is
`.ts`	**TypeScript**: JavaScript with labels saying what kind of value something is (`name: string` means "name is text").
`.tsx`	TypeScript that also contains **screen layout** written like HTML (`<button>Log in</button>`). This is React.
`.css`	How things **look**: colours, sizes, spacing.
`.sql`	Instructions for the **database**: which tables exist and what rules they follow.
`.json`	Plain **lists of settings** (e.g. the hashtags the Instagram reader watches).
`.md`	Written **documents** for humans (README, guides).

2. The folder map

Open the `comoonly` folder. Here's what everything at the top is for. **Bold** = where you'll spend your time.

Folder / file	What's inside
`src/`	The website (the screens).
`server/`	The server and its tests.
`docs/`	Guides: `RUNNING.md` (how to run and share), `DATABASE.md` (the data model, with a diagram).
`admin/index.html`	The starting page of your separate admin interface (`/admin/`).
`index.html`	The starting page of the public app.
`public/`	Files given out as-is (the browser-tab icon).
`package.json`	The project's ID card: its name, the add-on packages it uses, and the **commands** (`npm run live` , `npm test` …).
`.env`	**Your private settings and keys** (Gemini, Instagram, admin emails). Never uploaded. `.env.example` is the public template.
`README.md`	The project's front page on GitHub.
`vite.config.ts` , `tsconfig*.json` , `eslint.config.js` , `.prettierrc.json`	Tool settings (how the website is built, checked and formatted). You rarely need to open these.
`node_modules/` , `dist/` , `server/data/`	Made automatically: installed packages, the built website, the database. **Never edit by hand.**

Inside `server/`

```
server/
  src/
    index.ts      ← starts everything
    app.ts        ← list of all features
    config.ts     ← reads .env settings
    db/
      connection.ts  ← opens database
      migrations/    ← table blueprints
      seed.ts        ← sample data (tests)
    middleware/
      auth.ts    ← "who is this?"
      errors.ts  ← friendly errors
    modules/     ← one folder per feature
      auth/  people/  groups/
      requests/  chat/
      workspace/  events/
      notifications/  safety/
      admin/
  test/          ← automatic checks
```

Inside `src/`

```
src/
  main.tsx     ← starts the website
  App.tsx      ← address → screen
  api.ts       ← talks to the server
  types.ts     ← shapes of data
  App.css      ← how it looks
  pages/       ← one file per screen
    Discover, Groups, Create,
    Messages, Me, Events, …
  components/  ← reusable pieces
    GroupCard, Chat, Profile,
    UI (buttons, chips…), …
  state/       ← shared memory:
                 who's logged in, groups
  admin/       ← your /admin/ interface
  hooks/       ← small helpers
  assets/      ← logo, mascots, fonts
```

Tip: in VS Code or the Claude app's Files pane, click the little arrows to open and close folders. Start with the **bold** files: they're the doors into everything else.

3. The one pattern: three files per feature

Every folder in `server/src/modules/` is one feature, and (almost) every one has the same three kinds of file. Learn this once and you can read all of them.

File	Restaurant job	What it does
`groups.routes.ts`	Receptionist	Lists the **addresses** the feature answers (`GET /api/groups` , `POST /api/groups`). Reads the request, calls the service, sends back the answer. No decisions.
`groups.schema.ts`	Bouncer	Describes what **valid input** looks like ("a name, 1–70 letters"). Bad input is turned away with a friendly message before anything happens.
`groups.service.ts`	Chef	The **real work**: the rules ("only members can see members") and the database reads and writes.

So when you wonder…

- "**What can this feature do?**" → open its `.routes.ts` . Each `….get(` / `….post(` line is one action.
- "**Why was my input refused?**" → its `.schema.ts` .
- "**How does it actually decide?**" → its `.service.ts` .

What each feature folder is about

Folder	Feature
`auth`	Sign up, log in, log out, delete account, password hashing.
`people`	Profiles and connections between people.
`groups`	The group feed, one group's page, creating groups, saved groups, members.
`requests`	Asking to join a group, the host's accept/decline, leaving a group.
`chat`	Group chat and private messages.
`workspace`	Inside a group: activities ("I'm going") and to-dos.
`events`	The Events page and the event finder: `events.finder.ts` (web pages), `events.instagram.ts` (Instagram), `events.ai.ts` (Gemini reads posts). Settings: `sources.json` , `instagram.json` .
`notifications`	The bell and the Messages badge.
`safety`	Block and report.
`admin`	Your dashboard's data: totals, people, reports, backups.

The website side has a pattern too. A screen in `src/pages/` is built from pieces in `src/components/` , gets shared things (who's logged in, the groups list) from `useApp()` , and talks to the server only through `api(…)` or `useApi(…)` from `src/api.ts` . When you see `api('/groups/…')` in a screen, the matching code is in the server's `groups` (or `requests`) folder.

4. Follow one click: "Request to join"

Best way to understand code: pick one button and follow it all the way. Here's what happens when someone presses **Send join request**, in five stops. These are real lines from your files.

1. **The button**: `src/components/JoinAction.tsx`. The form calls `join(id, intro)`, which comes from the shared app state.

2. **The website asks the server**: `src/state/AppProvider.tsx`

```
// src/state/AppProvider.tsx
join: async (id, intro) => {
  const { status } = await api('/groups/' + id + '/requests', {
    method: 'POST',
    body: { intro },
  })
  setMembership((s) => ({ ...s, [id]: status }))  // remember "pending" so the screen
updates
```

   `POST` means "create something". `await` means "wait for the answer before going on".

3. **The receptionist**: `server/src/app.ts` sends every `/api/groups…` address to the groups and requests features. In `requests.routes.ts`:

```
// server/src/modules/requests/requests.routes.ts
requestRoutes.post('/:id/requests', requireAuth, (req, res) => {
  const { intro } = joinSchema.parse(req.body)         // bouncer checks the hello
  res.status(201).json({ status: requestToJoin(req.params.id, req.user.id, intro) })
})
```

   `:id` is a blank filled from the address (the group's id). `requireAuth` = "must be logged in". `201` = "created".

Continued on the next page: the bouncer and the chef.

4. Follow one click (continued)

4 **The bouncer**: `requests.schema.ts` says what a valid hello is:

```
// server/src/modules/requests/requests.schema.ts
intro: z.string().trim().min(1, 'Say a little hello first').max(300, 'Keep your hello under
300 characters')
```

5 **The chef**: `requests.service.ts` applies the rules, writes to the database and notifies the host:

```
// server/src/modules/requests/requests.service.ts
export function requestToJoin(groupId, userId, intro) {
  const group = findGroup(groupId)
  const status = statusOf(groupId, userId)
  if (status === 'declined') throw new HttpError(409, 'The host has declined this request')
  if (status) return status                                  // already waiting or already in
  if (group.spots === 0) throw new HttpError(409, 'This group is full')
  if (isBlocked(group.ownerId, userId)) throw new HttpError(403, 'You can't join this
group')
  db.prepare("INSERT INTO group_members (…) VALUES (?, ?, 'pending', ?)").run(groupId,
userId, intro)
  notify(group.ownerId, 'update', `${nameOf(userId)} asked to join ${group.name}`, …)
  return 'pending'
}
```

Read it like a recipe, top to bottom: each `if … throw` line is a rule that can stop the request with a message. `INSERT INTO` saves a row in the database table `group_members`.

The answer travels back the same way: service → route → website, and the button turns into "Pending · A hello in progress". If a rule said no, its message (e.g. "This group is full") is what the person sees on screen.

5. How to read any file

1. **Read the comments first.** Lines starting with `//` (or `--` in `.sql`) are notes for humans. In this project they explain *why* something is done; skim them before the code.
2. **Look at the `import` lines at the top.** They tell you what this file uses and from which file. `import { notify } from '../notifications/notifications.service.ts'` means "this file sends notifications".
3. **Find the `export`s.** `export function …` is something other files can use: the file's "public menu". Functions without `export` are private helpers.
4. **Read function names before function bodies.** Names like `requestToJoin`, `listMembers`, `deleteAccount` tell you the story. Only open the body when you need the details.
5. **Ignore what you don't need yet.** Labels after colons (`: string`, `: Group[]`) are TypeScript describing the data; you can mostly read past them.
6. **Search.** In VS Code press `Cmd+Shift+F` and type a word you see on screen ("This group is full"). You'll land on the exact line that shows it. This is the fastest way to find anything.

Reading a screen file (`.tsx`)

A screen is a function that **returns what to draw**. The part after `return (` looks like HTML: that's the layout. Above it are the screen's memory (`useState`) and actions (functions like `submit`).

```
// a tiny made-up screen, same shape as yours
export default function Hello() {
  const [count, setCount] = useState(0)      // the screen's memory: starts at 0
  return (
    <button onClick={() => setCount(count + 1)}>   // clicking changes the memory…
      Clicked {count} times                        // …and the text redraws by itself
    </button>
  )
}
```

`{ }` inside the layout means "insert a value here". `className="button lime"` picks styles from `App.css`. `{error && <p>…</p>}` means "show this only if there is an error".

Reading the database blueprint

`server/src/db/migrations/001_initial.sql` lists every table. Each `CREATE TABLE` block is one kind of thing (users, groups, messages…), and each line inside is one piece of information it keeps. `docs/DATABASE.md` explains them with a diagram. Read that first.

6. "Where do I look when…"

I want to…	Look in
Change words on a screen	That screen in `src/pages/` (or search the words with `Cmd+Shift+F`)
Change colours, sizes, spacing	`src/App.css` (colour names at the top of `src/index.css`)
Change a piece used on many screens (a card, chat box)	`src/components/`
See which address opens which screen	`src/App.tsx`
Change an error message ("Use at least 8 characters…")	The feature's `.schema.ts` (input rules) or `.service.ts` (other rules)
Change who's allowed to do something	The feature's `.service.ts`; admin-only: `requireAdmin` in `.routes.ts`
Change the hashtags the Instagram reader watches	`server/src/modules/events/instagram.json` (first 30 are used)
Change how the AI reads posts	The instructions to Gemini in `server/src/modules/events/events.ai.ts`
Change admin emails, AI or Instagram keys	`.env` (then restart the app)
Add a new piece of information to store	A **new** file `server/src/db/migrations/002_….sql`; never edit 001
Change your dashboard	`src/admin/` (screens) and `server/src/modules/admin/` (data)
See if everything still works	Run `npm test`; the checks live in `server/test/`

Commands you'll use (type them in Terminal, inside the folder)

Command	What it does
`npm run live`	Builds and starts the whole app on `http://localhost:3001` (keeps the Mac awake).
`npm run server` + `npm run dev`	Development mode, in two windows: changes appear as you save. Open `http://localhost:5173`.
`npm test`	Runs all automatic checks. "pass 18, fail 0" = all good.
`npm run format`	Tidies the spacing of all files.
`Ctrl+C`	Stops whatever is running in that Terminal window.

7. A small dictionary

Word	Meaning
API	The set of addresses (`/api/…`) the website uses to ask the server for things.
Route / endpoint	One of those addresses plus what it does, e.g. `POST /api/groups` = "create a group".
GET / POST / PATCH / DELETE	The kind of request: read / create / change / remove.
Status code	The server's short answer: `200` OK, `201` created, `400` bad input, `401` not logged in, `403` not allowed, `404` not found, `409` conflict (e.g. full).
Schema	A description of valid input (we use the Zod library for this).
Service	The file with a feature's rules and database work.
Middleware	A check that runs before a route, e.g. `requireAuth` ("must be logged in").
Database, table, row	The storage file; one kind of thing (users); one item of it (one user).
Migration	A numbered file that creates or changes tables. Runs once, in order.
Session / cookie	How you stay logged in: the server gives your browser a secret ticket (cookie) and remembers it (session).
Component	A reusable piece of screen (a card, a button row).
Props	Information you hand to a component, e.g. `<GroupCard group={g} />`.
State	A screen's memory that redraws the screen when it changes (`useState`).
async / await	"This takes time (e.g. asking the server); wait for the answer."
JSON	A plain text format for data: `{ "name": "Film club", "spots": 3 }`.
`.env`	Private settings and keys, kept out of GitHub.
Commit	A saved snapshot of the project in Git, with a message saying what changed.

8. Practice: three small exercises

1. Find a message. On the Create screen, the button says "Publish invitation →". Search for it with `Cmd+Shift+F`. Which file is it in? (Answer: `src/pages/Create.tsx`.)

2. Follow a rule. What stops someone joining a full group? Open `server/src/modules/requests/requests.service.ts` and find the line with `'This group is full'`. Which value is it checking? (Answer: `group.spots === 0`.)

3. Read a table. Open `server/src/db/migrations/001_initial.sql` and find `CREATE TABLE groups`. Which three colours can a group card be? (Answer: lilac, mint, pink, in the `color` line.)

Stuck on a file? Ask Claude: "explain *file name* line by line like I'm new to coding". That's a perfectly good way to learn.